



WILLSIM

FICHE DE COURS · RESSOURCE PÉDAGOGIQUE GRATUITE

# Maintenance prédictive : capteurs et détection d'anomalies

Écouter une machine tournante avant qu'elle ne tombe en panne : mesurer ses vibrations et son courant, en extraire des indicateurs fiables, puis détecter automatiquement un comportement anormal grâce à un modèle appris sur un fonctionnement sain.

## NIVEAU

Intermédiaire

## DURÉE

6 h (3 séances de 2 h)

## PRÉREQUIS

Le cours « Initiation à l'IA pour l'ingénierie »

## OUTILS

Accéléromètre, capteur de courant (INA219, ACS712...), Python, NumPy, SciPy, scikit-learn

## OBJECTIFS DE LA FICHE

À la fin de cette fiche, vous saurez :

- choisir et justifier une stratégie de maintenance (corrective, préventive ou prédictive) ;
- dimensionner une acquisition : fréquence d'échantillonnage (critère de Nyquist), durée de fenêtre, résolution ;
- calculer un spectre par FFT et y reconnaître un balourd (1×), un désalignement (2×), un défaut de roulement ;
- calculer et interpréter la valeur efficace (RMS), le facteur de crête et le kurtosis d'un signal ;
- entraîner une Isolation Forest sur un fonctionnement sain, régler son seuil d'après un taux de fausses alertes visé et confirmer les alertes par persistance.

## 1. Trois stratégies de maintenance

Une machine se dégrade progressivement : un roulement s'use, un accouplement se désaligne, un dépôt déséquilibre un ventilateur. La question n'est pas de savoir *si* il faudra intervenir, mais *quand*. Trois stratégies coexistent.

- **Corrective** : on répare après la panne. Pas de surveillance, mais des arrêts imprévus et des dégâts en cascade.
- **Préventive systématique** : on intervient selon un échéancier (heures, cycles, calendrier). Simple, mais des pièces saines sont changées trop tôt.
- **Prédictive** : on intervient selon l'état mesuré et son évolution prévue. Au bon moment, au prix de capteurs, de données et de compétences.

La norme NF EN 13306 range la maintenance systématique et la maintenance **conditionnelle**, déclenchée par l'état mesuré, dans la maintenance préventive ; elle appelle maintenance **prévisionnelle** (le terme français pour « prédictive ») la maintenance conditionnelle qui s'appuie en plus sur une prévision de la dégradation. Cette fiche suit la chaîne capteur → acquisition → indicateurs → modèle → alerte.

## 2. Capteurs et signatures de défauts

### 2.1 Choisir le capteur

Tableau 1 — Capteurs courants en atelier

| Capteur                                  | Grandeur mesurée                                         | Rapidité                                             | Usage typique                               |
|------------------------------------------|----------------------------------------------------------|------------------------------------------------------|---------------------------------------------|
| Accéléromètre MEMS (ADXL345, MPU-6050)   | Accélération 3 axes                                      | 1000 mesures/s (MPU-6050) à 3200 mesures/s (ADXL345) | Vibrations jusqu'à quelques centaines de Hz |
| Accéléromètre piézoélectrique industriel | Accélération (souvent 1 axe)                             | Bande de plusieurs kHz                               | Roulements, engrenages                      |
| INA219                                   | Courant (par shunt) et tension, liaison I <sup>2</sup> C | Conversion 12 bits en 532 µs                         | Tendance de consommation                    |
| ACS712                                   | Courant (effet Hall), sortie analogique                  | Capteur rapide (80 kHz), limité par le CAN           | Courant moteur, analyse spectrale           |

Un accéléromètre se fixe **rigidement** (vis, colle ou embase magnétique) près du palier, toujours au même endroit et dans la même orientation, radiale ou axiale : une mesure n'a de valeur que comparée aux précédentes.

### 2.2 Signatures vibratoires

Un arbre qui tourne à  $N$  tr/min a pour fréquence de rotation  $f_r = N/60$ , soit 25 Hz à 1500 tr/min. On repère les raies du spectre en **ordres**, c'est-à-dire en multiples de  $f_r$  : 1×, 2×, 3×...

Tableau 2 — Signatures typiques de trois défauts

| Défaut                     | Signature spectrale                                      | Indices complémentaires                                 |
|----------------------------|----------------------------------------------------------|---------------------------------------------------------|
| <b>Balourd</b>             | Raie dominante à 1×, surtout radiale                     | Force centrifuge proportionnelle au carré de la vitesse |
| <b>Désalignement</b>       | Raie marquée à 2×, souvent avec 1× et 3×                 | Vibration axiale élevée (désalignement angulaire)       |
| <b>Défaut de roulement</b> | Raies non entières (BPFO, BPFI, BSF, FTF) et harmoniques | Chocs répétés : facteur de crête et kurtosis en hausse  |

Les fréquences de défaut d'un roulement dépendent de sa géométrie : nombre d'éléments roulants  $n$ , diamètre des billes  $d$ , diamètre primitif  $D$ , angle de contact  $\varphi$ . Pour un défaut de bague extérieure (BPFO, *Ball Pass Frequency Outer race*) ou de bague intérieure (BPFI) :

$$\text{BPFO} = \frac{n}{2}f_r \left(1 - \frac{d}{D}\cos\varphi\right) \quad \text{BPFI} = \frac{n}{2}f_r \left(1 + \frac{d}{D}\cos\varphi\right)$$

BSF et FTF, elles, concernent les billes et la cage. Exemple : un roulement 6205 (9 billes,  $d = 7,94$  mm,  $D = 39,04$  mm,  $\varphi = 0$ ) donne BPFO = 3,585  $f_r$  et BPFI = 5,415  $f_r$ , soit 89,6 Hz et 135,4 Hz à 1500 tr/min. Ces raies ne tombent pas sur un ordre entier : c'est ce qui les distingue des défauts liés à la rotation (figure 1).

Sur un roulement peu dégradé, elles émergent toutefois mal du spectre brut : on les cherche alors dans le spectre de l'**enveloppe** du signal (démodulation), technique hors du cadre de cette fiche.

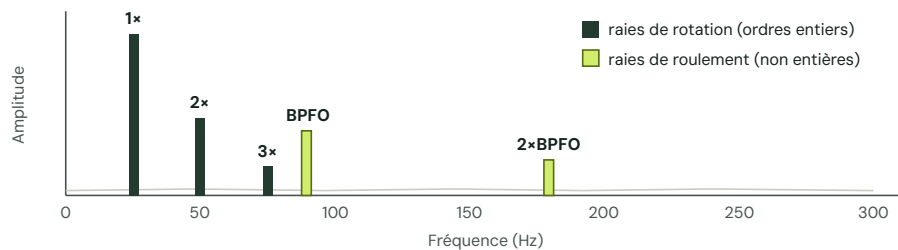


Figure 1 — Spectre schématisé d'un moteur à 1500 tr/min ( $f_r = 25$  Hz) : balourd, léger désalignement et défaut de bague extérieure d'un roulement 6205.

### 3. Échantillonnage et critère de Nyquist

D'après le théorème de Shannon-Nyquist, un signal dont le contenu ne dépasse pas  $f_{\max}$  est entièrement décrit par ses échantillons si la fréquence d'échantillonnage  $f_e$  vérifie :

$$f_e > 2f_{\max}$$

Sinon, une composante de fréquence  $f$  supérieure à  $f_e/2$  se **replie** : elle apparaît à la plus petite des fréquences  $|f - k f_e|$ . Échantillonnée à 1000 Hz, une vibration à 900 Hz se lit à 100 Hz, sans moyen de la reconnaître après coup. On prévient donc le repliement *avant* la conversion : filtre passe-bas analogique devant le CAN, ou capteur à bande passante limitée en interne, comme beaucoup d'accéléromètres MEMS. En pratique, on échantillonne au moins 2,5 fois plus vite que la plus haute fréquence utile.

La durée d'observation  $T = N/f_e$  fixe la **résolution fréquentielle**, l'écart entre deux points du spectre :  $\Delta f = f_e/N = 1/T$ . Pour notre moteur, les raies utiles (rotation, BPFO, BPFI et leurs premières harmoniques) restent sous 600 Hz. On retient  $f_e = 1600$  Hz, l'un des débits de l'ADXL345 (qui exige alors une liaison SPI rapide), et des fenêtres de  $N = 1600$  échantillons :  $T = 1$  s et  $\Delta f = 1$  Hz.

### 4. Lire un spectre avec la FFT

La transformée de Fourier discrète décompose une fenêtre de  $N$  échantillons en raies de fréquences  $f_k = k f_e/N$  ; la FFT (*Fast Fourier Transform*) la calcule rapidement. Pour un signal réel, `np.fft.rfft` renvoie les  $N/2 + 1$  raies de 0 à  $f_e/2$  (pour  $N$  pair).

spectre.py

```
import numpy as np
from scipy.signal import find_peaks

fe, N, fr = 1600, 1600, 25.0          # 1600 Hz, 1 s de signal, rotation à 1500 tr/min
t = np.arange(N) / fe
rng = np.random.default_rng(0)
# Signal synthétique (m/s²) : balourd (1x) + Léger désalignement (2x) + bruit
x = 0.8 * np.sin(2 * np.pi * fr * t) + 0.3 * np.sin(2 * np.pi * 2 * fr * t)
x += 0.1 * rng.standard_normal(N)
w = np.hanning(N)                    # fenêtre de Hann : Limite la fuite spectrale
A = 2 * np.abs(np.fft.rfft(x * w)) / w.sum() # amplitude crête de chaque raie
f = np.fft.rfftfreq(N, d=1 / fe)      # de 0 à fe/2 par pas de fe/N = 1 Hz
pics, _ = find_peaks(A, height=0.1)   # raies qui dépassent 0,1 m/s²
for k in pics:
    print(f"{f[k]:5.1f} Hz    amplitude {A[k]:.2f}    ({f[k] / fr:.0f}x)")
```

```
25.0 Hz  amplitude 0.80  (1x)
50.0 Hz  amplitude 0.30  (2x)
```

Deux précautions dans ce code : la **fenêtre de Hann** limite l'étalement d'une raie sur ses voisines (fuite spectrale) ; la **normalisation** par la somme de ses poids donne directement l'amplitude crête de chaque sinusoïde. On retrouve ainsi les amplitudes simulées. Sur une vraie machine, on moyenne les spectres de plusieurs fenêtres pour lisser le fond de bruit (méthode de Welch, `scipy.signal.welch`).

## 5. Indicateurs temporels

Trois nombres résument une fenêtre  $x_1 \dots x_N$  centrée (moyenne retirée), d'écart-type  $\sigma$  :

$$\text{RMS} = \sqrt{\frac{1}{N} \sum_{i=1}^N x_i^2} \quad \text{FC} = \frac{\max |x_i|}{\text{RMS}} \quad K = \frac{\frac{1}{N} \sum_{i=1}^N x_i^4}{\sigma^4}$$

- La **valeur efficace (RMS)** mesure l'énergie vibratoire globale et réagit bien au balourd et au désalignement. Dans l'industrie, la sévérité vibratoire s'évalue en vitesse efficace (mm/s) selon la série de normes ISO 20816.
- Le **facteur de crête (FC)** compare la valeur de crête à la valeur efficace :  $\sqrt{2} \approx 1,41$  pour une sinusoïde, davantage avec des chocs.
- Le **kurtosis (K)** mesure le caractère impulsif : 1,5 pour une sinusoïde, 3 pour un bruit gaussien, plus avec des chocs. Il signale tôt un roulement qui s'abîme, mais redescend quand les chocs deviennent permanents : suivez-le dans le temps.

Tableau 3 — Indicateurs calculés sur 1 s de signal échantillonné à 1600 Hz

| Signal                                                                             | RMS   | FC   | K    |
|------------------------------------------------------------------------------------|-------|------|------|
| Sinusoïde d'amplitude 1 à 25 Hz                                                    | 0,707 | 1,41 | 1,50 |
| Même sinusoïde + un choc d'amplitude 3 tous les 18 échantillons ( $\approx 89$ Hz) | 0,986 | 3,89 | 5,68 |
| Bruit gaussien d'écart-type 1 (un tirage)                                          | 0,987 | 3,94 | 3,12 |

Le bruit et le signal à chocs ont presque le même facteur de crête, mais pas le même kurtosis : aucun indicateur ne suffit seul. La fonction `caracteristiques` de `detection.py` (paragraphe 6.2) calcule ces trois indicateurs. Attention à `fisher=False` : par défaut, `scipy.stats.kurtosis` renvoie le kurtosis *en excès* ( $K - 3$ , nul pour un bruit gaussien).

## 6. Détecter les anomalies avec une Isolation Forest

### 6.1 Apprendre le fonctionnement sain

Les pannes sont rares et variées : on manque d'exemples de chaque défaut pour entraîner un classifieur. On apprend donc le fonctionnement **normal**, puis on signale tout ce qui s'en écarte : c'est la **détection de nouveautés**.

L'Isolation Forest construit de nombreux arbres aléatoires : à chaque nœud, une caractéristique est tirée au hasard, puis un seuil entre le minimum et le maximum de cette caractéristique sur les données du nœud. Un point éloigné des autres est isolé après peu de découpes ; avec `scikit-learn`, `score_samples` renvoie donc

une valeur d'autant plus **basse** que le point est anormal. Inutile de normaliser : les seuils sont tirés dans l'intervalle propre à chaque caractéristique.

## 6.2 Mise en œuvre sur des données synthétiques

Chaque fenêtre de 1 s est résumée par cinq caractéristiques : RMS, facteur de crête, kurtosis, raies 1x et 2x. Le modèle n'apprend que des fenêtres saines ; le seuil est le 1<sup>er</sup> centile des scores d'autres fenêtres saines, gardées pour la validation.

detection.py

```
import numpy as np
from scipy.stats import kurtosis
from sklearn.ensemble import IsolationForest

fe, N, fr = 1600, 1600, 25.0          # fenêtres de 1 s, résolution de 1 Hz
t = np.arange(N) / fe
w = np.hanning(N)
rng = np.random.default_rng(1)
choc = np.exp(-300 * t[:40]) * np.sin(2 * np.pi * 500 * t[:40]) # résonance amortie

def fenetre(a1=0.5, a2=0.1, chocs=0.0): # 1 s d'accélération synthétique (m/s²)
    a1 *= rng.uniform(0.9, 1.1)        # dispersion normale d'une mesure à l'autre
    p = rng.uniform(0, 2 * np.pi, 2)
    x = a1 * np.sin(2 * np.pi * fr * t + p[0]) + a2 * np.sin(4 * np.pi * fr * t + p[1])
    impacts = np.zeros(N)
    impacts[(np.arange(0, 1, 1 / 89.6) * fe).astype(int)] = chocs # BPF0 = 89,6 Hz
    x += np.convolve(impacts, choc)[:N]
    return x + 0.1 * rng.standard_normal(N)

def caracteristiques(x):                # RMS, facteur de crête, kurtosis, raies 1x et 2x
    x = x - x.mean()
    rms = np.sqrt(np.mean(x ** 2))
    A = 2 * np.abs(np.fft.rfft(x * w)) / w.sum() # ici A[k] = raie à k Hz
    return [rms, np.max(np.abs(x)) / rms, kurtosis(x, fisher=False), A[25], A[50]]

sain = np.array([caracteristiques(fenetre()) for _ in range(500)])
modele = IsolationForest(n_estimators=200, random_state=0).fit(sain[:300]) # sain seul
seuil = np.percentile(modele.score_samples(sain[300:]), 1) # vise 1 % de fausses alertes

cas = {"sain": {}, "balourd": {"a1": 0.8}, "désalignement": {"a2": 0.3}, "roulement": {"chocs": 0.5}}
for nom, params in cas.items():
    X = np.array([caracteristiques(fenetre(**params)) for _ in range(100)])
    taux = np.mean(modele.score_samples(X) < seuil) # score bas = anormal
    print(f"{nom:14s} fenêtres signalées : {taux:4.0%}")
```

Sortie

```
sain          fenêtres signalées :  1%
balourd       fenêtres signalées : 100%
désalignement fenêtres signalées : 100%
roulement    fenêtres signalées : 86%
```

## 6.3 Limites à connaître

- Le modèle dit « différent », pas « quel défaut » : le diagnostic se fait avec le spectre et le tableau 2.
- Le score **sature** hors du domaine appris : un point très éloigné n'est pas jugé plus anormal qu'un point situé juste au-delà des données saines. Gardez des seuils absolus (RMS maximale, par exemple).
- Vitesse et charge modifient les signaux sans qu'il y ait de défaut : apprenez sur tous les régimes normaux ou ajoutez la vitesse aux caractéristiques. Après une intervention, réapprenez la référence saine.

## 7. Seuils et alertes

Avec un seuil au 1<sup>er</sup> centile, une fenêtre saine sur cent environ est signalée : une anomalie isolée ne suffit pas à alerter.

- **Persistance** : alerter après  $k$  fenêtres anormales consécutives (ou  $k$  parmi les  $n$  dernières). Si 1 % des fenêtres saines sont signalées, de façon indépendante, trois de suite n'arrivent qu'avec une probabilité de  $10^{-6}$  ; à raison d'une fenêtre par seconde, cela fait encore une fausse alerte tous les 12 jours environ.
- **Deux niveaux et hystérésis** : « vigilance » si l'anomalie est isolée, « alerte » si elle persiste ; on ne lève l'alerte qu'après plusieurs fenêtres normales.
- **Traçabilité** : enregistrer les signaux bruts autour de chaque alerte pour le diagnostic.

## 8. Du PC vers le microcontrôleur

Le PC sert à collecter les données, choisir les caractéristiques, entraîner le modèle et régler le seuil. Le microcontrôleur acquiert le signal et ne transmet que les indicateurs de chaque fenêtre, ce qui soulage la liaison, le stockage et la batterie. Exemple : le courant d'un petit moteur à courant continu, mesuré par un module ACS712 (185 mV/A en version 5 A, sortie à mi-tension à courant nul) relié à un Arduino Uno. À vitesse égale, une hausse du courant moyen traduit un couple résistant plus élevé : frottements, grippage, roulement qui se dégrade.

- **Cadence fixe** : déclenchez chaque mesure sur une horloge (`micros()` ou, mieux, un timer matériel), jamais « aussi vite que possible » : la FFT suppose des échantillons régulièrement espacés.
- **Calibration et résolution** : mesurez la sortie à courant nul au démarrage, moteur à l'arrêt (elle s'écarte un peu de 2,5 V d'un module à l'autre). Avec le CAN 10 bits sous 5 V, un pas vaut 4,9 mV, soit environ 26 mA.
- **Mémoire** : l'Uno n'a que 2 Ko de RAM. Cumulez au fil de l'eau les sommes de  $i$  et de  $i^2$  : elles donnent moyenne et valeur efficace sans stocker la fenêtre.
- **Repliement** : si le moteur est piloté en MLI (PWM), l'ondulation à la fréquence de découpage peut se replier dans la bande mesurée : filtre RC passe-bas devant l'entrée analogique.

Pour calculer un spectre sur la carte, un microcontrôleur ARM Cortex-M et la bibliothèque CMSIS-DSP (FFT optimisées) conviennent mieux. Exécuter le modèle de détection sur la carte elle-même est l'objet du cours « IA embarquée sur microcontrôleur (TinyML) ».

### ATTENTION · SÉCURITÉ

- Machine tournante : carter de protection en place, ni vêtements amples, ni cheveux longs détachés, ni bijoux. Fixez ou déplacez un capteur uniquement machine **à l'arrêt et consignée** (alimentation coupée et condamnée).
- Balourd volontaire : masse vissée et freinée, vitesse limitée, personne dans le plan de rotation.
- Très basse tension uniquement (moteurs de 24 V au plus), alimentation de laboratoire limitée en courant, fusible. Jamais de montage de prototypage (platine d'essai, module à bornier) sur le secteur 230 V.
- Choisissez la version de l'ACS712 d'après le courant de démarrage et de blocage du moteur, bien supérieur au courant nominal.

### À RETENIR

- Maintenance prédictive : intervenir au bon moment, d'après l'état mesuré.
- Balourd : 1×. Désalignement : 2×. Roulement : raies non entières et chocs (kurtosis).
- $f_e > 2 f_{\max}$ , filtre anti-repliement avant le CAN,  $\Delta f = 1/T$ .
- Isolation Forest apprise sur du sain, seuil fixé par le taux de fausses alertes, persistance, seuils absolus.

## 9. Exercice guidé

### EXERCICE · SURVEILLER UNE DÉRIVE DE BALOURD

À partir de `spectre.py` et de `detection.py` (environ 1 h, en fin de séance 3) :

1. Calculez le spectre de `fenetre(a2=0.3)` et relevez les amplitudes des raies à 25 Hz et 50 Hz.
2. Générez 200 nouvelles fenêtres saines et calculez la proportion signalée par le modèle.
3. Programmez une alerte déclenchée après 3 fenêtres anormales consécutives.
4. Simulez une dégradation : `fenetre(a1=0.5 * (1 + i / 100))` pour  $i$  de 0 à 99. Quand l'alerte se déclenche-t-elle, et de combien l'amplitude 1× a-t-elle augmenté ?
5. Bonus : ajoutez un seuil absolu (1,5 fois la RMS saine maximale). Pourquoi complète-t-il le modèle ?

#### Critères de réussite :

- raie à 50 Hz proche de 0,3 ; raie à 25 Hz entre 0,45 et 0,55 environ (dispersion de  $\pm 10\%$ ) ;
- au plus 5 % de fenêtres saines signalées et aucune alerte sur les 200 ;
- alerte avant +50 % sur l'amplitude 1× (typiquement +15 à +35 %), et persistance expliquée.

**Indice :** un compteur suffit. Pour chaque fenêtre, `compteur = compteur + 1 if anomalie else 0`, puis alerte dès que `compteur >= 3`. Pour le bonus, relisez la saturation du score décrite au paragraphe 6.3.

## Pour aller plus loin

- [NumPy : transformée de Fourier discrète \(numpy.fft\)](https://numpy.org/doc/stable/reference/routines.fft.html) — <https://numpy.org/doc/stable/reference/routines.fft.html>
- [SciPy : méthode de Welch](https://docs.scipy.org/doc/scipy/reference/generated/scipy.signal.welch.html) — <https://docs.scipy.org/doc/scipy/reference/generated/scipy.signal.welch.html>
- [scikit-learn : détection de nouveautés et d'anomalies](https://scikit-learn.org/stable/modules/outlier_detection.html) — [https://scikit-learn.org/stable/modules/outlier\\_detection.html](https://scikit-learn.org/stable/modules/outlier_detection.html)
- [scikit-learn : IsolationForest](https://scikit-learn.org/stable/modules/generated/sklearn.ensemble.IsolationForest.html) — <https://scikit-learn.org/stable/modules/generated/sklearn.ensemble.IsolationForest.html>
- [Allegro Microsystems : ACS712](https://www.allegromicro.com/en/products/sense/current-sensor-ics/integrated-current-sensors/acs712) — <https://www.allegromicro.com/en/products/sense/current-sensor-ics/integrated-current-sensors/acs712>
- [Page du cours sur willsim.fr](https://willsim.fr/cours/maintenance-predictive.html) — <https://willsim.fr/cours/maintenance-predictive.html>

**Fiche rédigée par l'équipe pédagogique WILLSIM**, association loi 1901 (RNA W842013616) qui promeut l'usage de l'intelligence artificielle dans l'ingénierie et la mécanique. Version 1.0 – octobre 2026.

© 2026 WILLSIM — Licence Creative Commons BY-NC-SA 4.0 ([creativecommons.org/licenses/by-nc-sa/4.0/deed.fr](https://creativecommons.org/licenses/by-nc-sa/4.0/deed.fr)) : vous pouvez copier, diffuser et adapter cette fiche à des fins non commerciales, en citant WILLSIM et en partageant vos adaptations sous la même licence.

Une erreur, une suggestion ? Écrivez à [contact@willsim.fr](mailto:contact@willsim.fr) · [willsim.fr](https://willsim.fr)